```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
```

```
# --- 1. Load and preprocess dataset ---
file_path = 'https://drive.google.com/uc?id=1jimNI-pY7u4tDS3sNqeucDbc0lNelJKg'
df = pd.read_csv(file_path, encoding='ISO-8859-1')
```

```
df.head()
```

	Date	Article_title	Stock_symbol	Sentiment	Confidence_Score	Change
0	1/2/2018	How The Parts Add Up: XWEB Targets $77	EXPE	Positive	0.870206	0.0
1	1/2/2018	North Korean Leader Kim Jong Un Mentions Possi...	EWY	Positive	0.831472	0.0
2	1/2/2018	Rite Aid Stock Has a Lot to Prove on Wednesday	WBA	Positive	0.845923	0.0
3	1/2/2018	Macau's Gaming Inspection and Coordination Bur...	MPEL	Negative	0.454896	0.0

```
# Convert date and sentiment
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df['Sentiment_Num'] = df['Sentiment'].map({'Positive': 2, 'Neutral': 1, 'Negative': 0})
```

```
# Tokenize news headlines
tokenizer = Tokenizer(num_words=10000, oov_token="<OOV>")
tokenizer.fit_on_texts(df['Article_title'])
text_seq = tokenizer.texts_to_sequences(df['Article_title'])
text_pad = pad_sequences(text_seq, maxlen=30, padding='post', truncating='post')
```

```
# Combine features: headline (padded), sentiment label, and confidence score
sentiment_array = np.array(df['Sentiment_Num']).reshape(-1, 1)
confidence_array = np.array(df['Confidence_Score']).reshape(-1, 1)
X = np.hstack((text_pad, sentiment_array, confidence_array))
```

```
# Use actual price change as regression target
y = df['Change'].values
```

```
# --- 2. Train-test split ---
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# --- 3. Build LSTM regression model ---
model = Sequential([
    Embedding(input_dim=10000, output_dim=64, input_length=30),
    LSTM(64),
    Dropout(0.5),
    Dense(1, activation='linear')  # ✅ Regression output
])
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input_length` is deprecated. Just
  warnings.warn(
```

```
# --- 4. Compile model for regression ---
model.compile(optimizer='adam', lo
```

What can I help you build?

```
# --- 5. Train the model ---
model.fit(X_train, y_train, epochs=5, batch_size=32, validation_data=(X_test, y_test), verbose=1)
```

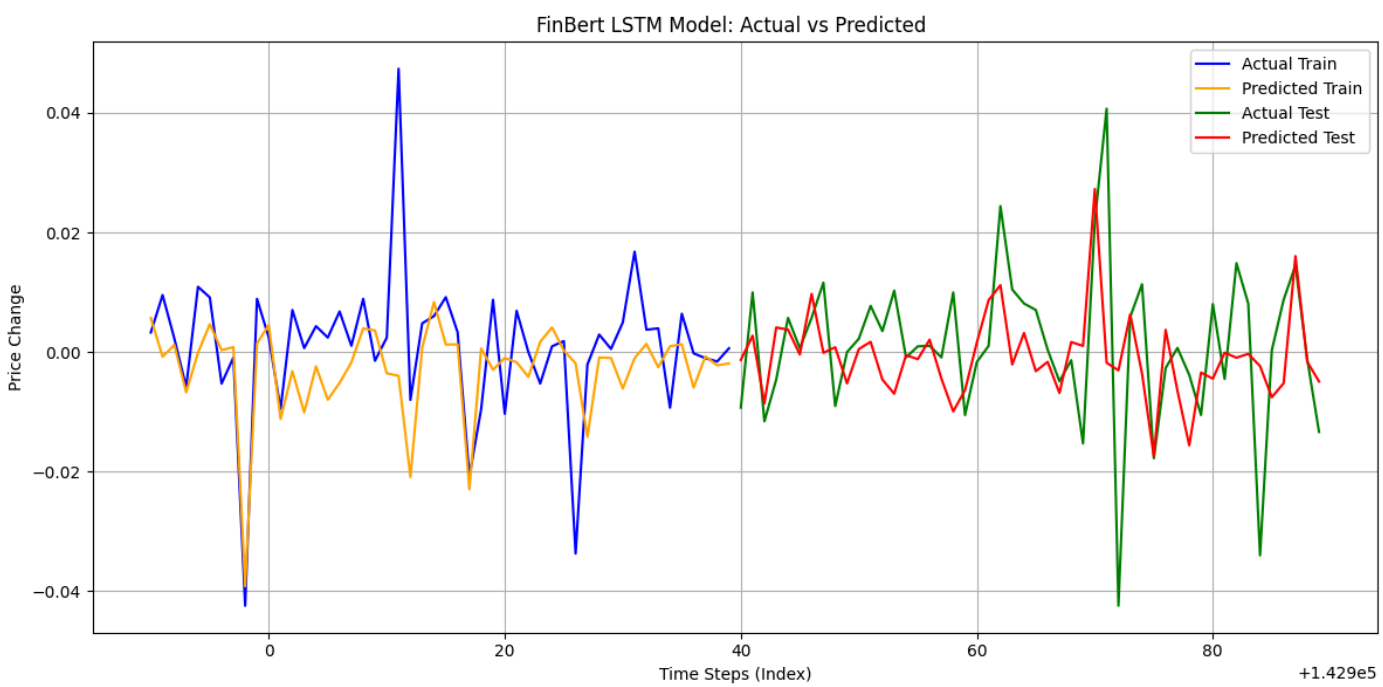
```
Epoch 1/5
4467/4467 ━━━━━━━━━━━━━━━━━━━━ 137s 30ms/step - loss: 1.7005e-04 - mae: 0.0092 - val_loss: 1.3801e-04 - val_mae: 0.0083
Epoch 2/5
4467/4467 ━━━━━━━━━━━━━━━━━━━━ 146s 31ms/step - loss: 1.4014e-04 - mae: 0.0082 - val_loss: 1.3011e-04 - val_mae: 0.0078
Epoch 3/5
4467/4467 ━━━━━━━━━━━━━━━━━━━━ 134s 29ms/step - loss: 1.3180e-04 - mae: 0.0080 - val_loss: 1.2811e-04 - val_mae: 0.0078
Epoch 4/5
4467/4467 ━━━━━━━━━━━━━━━━━━━━ 150s 31ms/step - loss: 1.2122e-04 - mae: 0.0077 - val_loss: 1.2408e-04 - val_mae: 0.0077
Epoch 5/5
4467/4467 ━━━━━━━━━━━━━━━━━━━━ 139s 31ms/step - loss: 1.1553e-04 - mae: 0.0075 - val_loss: 1.2206e-04 - val_mae: 0.0077
<keras.src.callbacks.history.History at 0x7df78e2c0650>
```

```
# --- 6. Predict on train and test sets ---
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
```

```
4467/4467 ━━━━━━━━━━━━━━━━━━━━ 32s 7ms/step
1915/1915 ━━━━━━━━━━━━━━━━━━━━ 13s 7ms/step
```

```
# --- 7. Evaluate performance ---
mae_train = mean_absolute_error(y_train, y_train_pred)
mse_train = mean_squared_error(y_train, y_train_pred)
rmse_train = np.sqrt(mse_train)


mae_test = mean_absolute_error(y_test, y_test_pred)
mse_test = mean_squared_error(y_test, y_test_pred)
rmse_test = np.sqrt(mse_test)


# --- 8. Display results as a table ---
metrics_table = pd.DataFrame({
    'Metric': ['MAE', 'MSE', 'RMSE'],
    'Train Set': [mae_train, mse_train, rmse_train],
    'Test Set': [mae_test, mse_test, rmse_test]
})

print(metrics_table.round(6))
```

```
  Metric  Train Set  Test Set
0    MAE   0.007158  0.007708
1    MSE   0.000105  0.000122
2   RMSE   0.010266  0.011048
```

```
import matplotlib.pyplot as plt
import numpy as np

# Step 1: Create x-axis indices
train_dates = np.arange(len(y_train))
test_dates = np.arange(len(y_train), len(y_train) + len(y_test))

# Step 2: Flatten predictions
train_pred = y_train_pred.flatten()
test_pred = y_test_pred.flatten()

# Step 3: Define zoom range (last 50 of train, first 50 of test)
zoom_start = len(train_dates) - 50
zoom_end = len(train_dates) + 50

# Step 4: Plot
plt.figure(figsize=(12, 6))

# Plot actual vs predicted (zoomed-in only)
plt.plot(train_dates[zoom_start:], y_train[zoom_start:], label='Actual Train', color='blue')
plt.plot(train_dates[zoom_start:], train_pred[zoom_start:], label='Predicted Train', color='orange')
plt.plot(test_dates[:50], y_test[:50], label='Actual Test', color='green')
plt.plot(test_dates[:50], test_pred[:50], label='Predicted Test', color='red')

# Styling
plt.title("FinBert LSTM Model: Actual vs Predicted ")
plt.xlabel("Time Steps (Index)")
plt.ylabel("Price Change")
plt.legend()
plt.grid(True)
plt.tight_layout()
```

```
plt.tight_layout()
plt.show()
```